

★ SPECIAL · PART 2

Advanced Recommenders

Factorization Machines to deep learning — NCF, Two-Tower, SASRec, GNN, LLM

Part 1 covered the classics. Part 2 climbs the ladder into deep learning, starting with the bridge — Factorization Machines — and ending at graph and language-based recommenders.

Quick Recap — Where We Left Off

The classics, and their ceiling

Part 1 gave us

- Content-based: match items by features.
- Collaborative filtering: match by who liked what.
- Matrix factorization: hidden 'taste dials' ($R = U V$).
- Honest evaluation; the cold-start problem.

...but the classics hit limits

- MF only uses user & item IDs — ignores context.
- Dot products capture only linear taste.
- No sense of ORDER (what you bought last).
- Struggle with huge, sparse, fast-moving catalogs.

Deep learning lifts each ceiling — but we climb there one rung at a time.

Where We Are Going Today

Four sections, one ladder

- 1 The bridge: Factorization Machines**
FM generalizes MF to all features; DeepFM bolts on a neural net. The step from classic to deep.
- 2 The deep recommenders**
NCF (non-linear), Two-Tower (scalable), SASRec (sequence-aware).
- 3 The graph & language frontier**
LightGCN and ContextGNN on the customer-product graph; LLM-based explainable recs.

Everything builds on Week 8 (neural nets), Week 9 (factors), Week 12 (attention, embeddings, LLM), Week 15 (GNN).



1

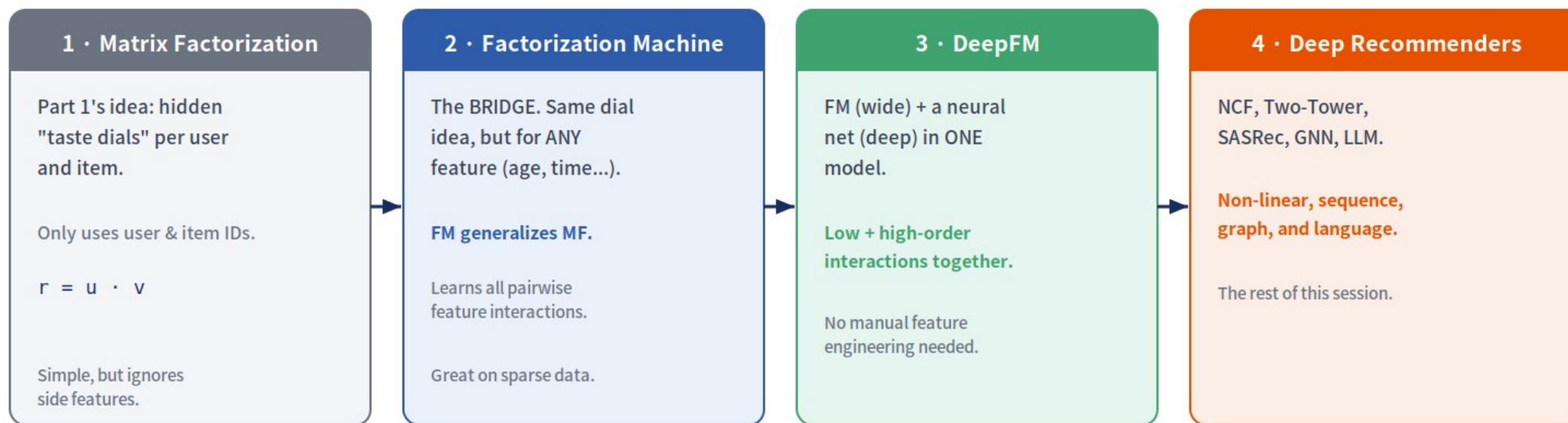
The Bridge

Factorization Machines and DeepFM

The Ladder From MF to Deep

Each rung adds power; FM is the crucial middle step

The Ladder — From Classic MF to Deep Recommenders



Each rung adds power: more features, then non-linearity, then sequence, graph, and language.

Don't jump straight to deep nets — understanding FM makes the deep models click.

Why We Need a Bridge

MF is powerful but narrow — FM widens it

Matrix factorization only knows two things: which user, which item. That's it.
But whether you'll like a fund depends on your age, the time, your goals — context MF simply can't see.

The problem

- Real decisions use many features, not just IDs.
- Hand-coding every feature pair is hopeless.
- And most feature pairs are rarely seen (sparse).

FM's promise

- Give EVERY feature its own hidden vector.
- Automatically learn how every pair interacts.
- Works even when data is very sparse.

Factorization Machines — Dials for Everything

FM = matrix factorization, generalized to all features

Factorization Machine — Dials for Every Feature, Not Just IDs

Matrix Factorization

Only two things have dials:

user ID

item ID

It can't use age, time of day,
device, or any side info.

Blind to context.

Factorization Machine

EVERY feature gets its own dial vector:

user

item

age

time

device

Then it learns how every PAIR interacts
(young user × evening × mobile → likes short clips).

**FM = MF generalized to all features — and it shines
on SPARSE data, where most pairs are rarely seen.**

'MF is a special case of FM.' Same dial idea, but now every feature joins in.

Factorization Machine, in Symbols

Looks scary, reads simple

$$\hat{y} = w_0 + \sum_i w_i x_i + \sum_{i < j} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

Reading it

- First term: a baseline score.
- Second term: each feature's own weight (like linear regression).
- Third term: the magic — every PAIR of features interacts, scored by the dot product of their hidden vectors.

Why it scales

- Naively, all pairs would be $O(\text{features squared})$ — too slow.
- A clever rewrite makes it $O(\text{features})$ — linear time.
- That's why FM works on huge, sparse ad/rec data.
- It's still the backbone of many ranking systems.

DeepFM — FM Meets Deep Learning

Wide and deep, sharing one input

Why choose? DeepFM runs an FM 'wide' part and a neural-net 'deep' part side by side.

FM catches low-order pairs; the deep net catches complex high-order patterns. They share the same inputs.

The two halves

- Wide (FM): 1st- and 2nd-order feature interactions.
- Deep (MLP): high-order, non-linear patterns.
- Combined for the final prediction (e.g. click probability).

Why it caught on

- No manual feature engineering needed.
- Trained end to end in one model.
- A workhorse for click-through-rate prediction.

2

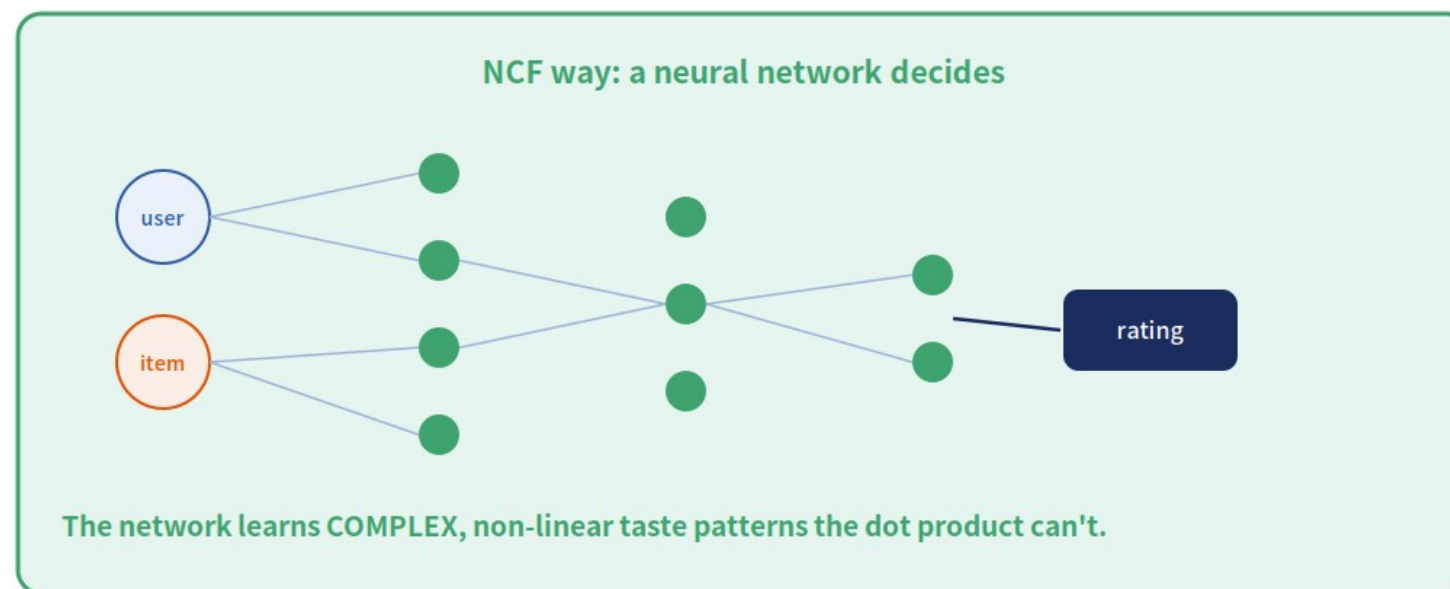
The Deep Recommenders

NCF, Two-Tower, and Sequential

NCF — Neural Collaborative Filtering

Swap the dot product for a neural network

NCF — Replace the Dot Product With a Neural Net



The dot product assumes linear taste; a network learns the curves and corners it misses.

NCF, in One Line

Feed the two vectors into an MLP

$$\hat{r}_{u,i} = \text{MLP}([\mathbf{u}_u \parallel \mathbf{v}_i])$$

Reading it

- Take the user vector and item vector.
- Glue them together (the $\parallel$ means concatenate).
- Feed them through a small neural network (MLP).
- Out comes the predicted rating or click.

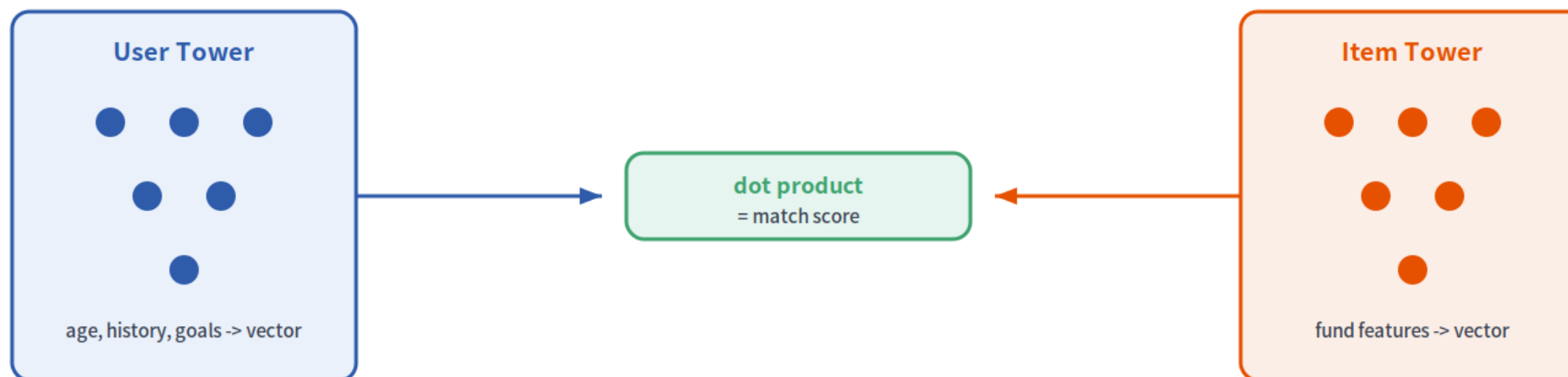
In finance

- Great for implicit feedback: bought vs not bought.
- Captures 'this combo of traits likes this product'.
- More flexible than plain matrix factorization.
- The natural first deep step beyond MF.

Two-Tower — Built for Scale

A user tower and an item tower, matched by dot product

Two-Tower — Build Two Embeddings, Match by Dot Product



Why it scales to millions

Item vectors are computed ONCE and stored. At request time, only the user vector is fresh — then a fast **nearest-neighbor search (Faiss)** finds the top matches among millions of items in milliseconds.

Precompute item vectors once; serve millions of customers in real time with fast nearest-neighbor search.

Two-Tower, and the Two-Stage System

Retrieve fast, then rank carefully

$$\text{score}(u, i) = f_{\text{user}}(u) \cdot g_{\text{item}}(i)$$

How it's trained & served

- Each tower is a neural net turning inputs into a vector.
- Trained with in-batch negatives (contrastive learning).
- Item vectors are stored; the user vector is computed live.
- Faiss finds the top matches among millions, in ms.

Two-stage recommendation

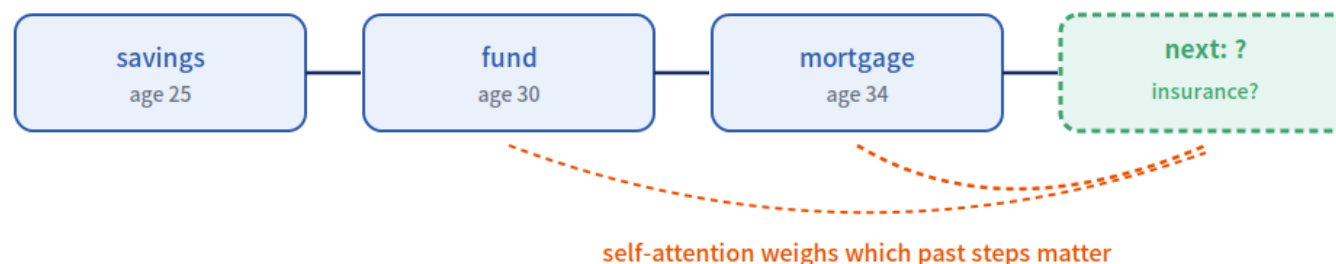
- Stage 1 — RETRIEVAL: Two-Tower narrows millions to a few hundred.
- Stage 2 — RANKING: a richer model (DeepFM, etc.) orders them.
- Towers can embed sequences (SASRec) or graphs (GNN).
- This is the standard industrial design.

SASRec — Sequence-Aware Recommendation

Self-attention over the order of what you did

SASRec — Read the Sequence, Predict What's Next

A customer's financial life, in order:



Order matters — and the Transformer (Week 12) reads it

SASRec uses self-attention over the PURCHASE SEQUENCE to predict the next product, learning life-stage patterns.

On dense histories it acts long-range (like an RNN); on sparse ones it leans on recent steps (like a Markov chain).

Time-aware variants (TiSASRec) even use the gaps between events.

Order carries meaning: savings then mortgage hints at the next life stage. SASRec reads that.

Why Order Matters in Finance

Your last move predicts your next one

Most recommenders treat your history as an unordered bag of items. But sequence is signal.

Opening a mortgage right after a home purchase points to insurance next — order reveals the life stage.

How SASRec works

- Self-attention (Week 12) over the purchase sequence.
- Weighs which past steps matter for the next one.
- Predicts the next product the customer will need.

Smart adaptivity

- Dense history: looks far back (like an RNN).
- Sparse history: leans on recent steps (Markov-like).
- Time-aware variants use the GAPS between events.

3

The Frontier

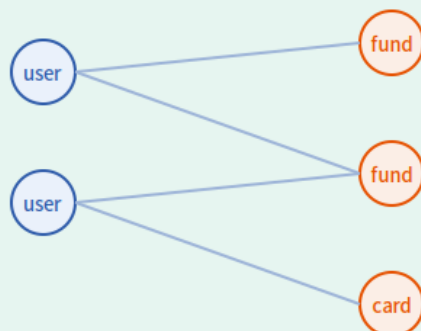
Graph and language recommenders

Graph & Language Recommenders

LightGCN on the customer-product graph; LLMs that explain

The Frontier — Graph and Language Recommenders

LightGCN — graph propagation



Customers and products form a bipartite graph.

Embeddings spread along the edges (no heavy layers).

Simple, strong on SPARSE financial data; learns high-order links.

LLM-based — explainable recs

An LLM (Week 12) can recommend AND explain WHY, in plain language.

"Based on your recent home purchase, consider mortgage insurance because..."

ContextGNN (ICLR 2025): hybrid of Two-Tower + GNN over relational data.

No single model wins everywhere — hybrids lead.

The newest ideas connect straight back to Week 15's GNNs and Week 12's LLMs.

LightGCN — Recommendation on a Graph

Customers and products as one big network

$$\mathbf{e}_v^{(k+1)} = \sum_{u \in N(v)} \frac{1}{\sqrt{|N(v)| |N(u)|}} \mathbf{e}_u^{(k)}$$

Reading it

- Build a graph: users link to products they used.
- A node's new vector = a clean average of its neighbors'.
- 'Light' = no heavy layers, just propagation (Week 15 idea).

Why finance likes it

- Strong on SPARSE data — most customers use few products.
- Learns high-order links (friends-of-friends of products).
- Surfaces non-obvious cross-sell connections.
- Simple and stable to train.

The Newest Ideas

Hybrids, and recommenders that explain themselves

ContextGNN (ICLR 2025)

- 'Beyond Two-Tower': blends Two-Tower with a GNN.
- Built for relational databases (many linked tables).
- Argues no single model wins everywhere — hybrids do.
- Frontier research, directly on the customer-product graph.

LLM-based recommendation

- An LLM (Week 12) recommends AND explains in words.
- 'Given your home purchase, consider insurance because...'
- Naturally handles cold-start using descriptions.
- But: weaker at long histories than SASRec — pair them.

The pattern: combine specialists. Retrieval, ranking, sequence, graph, and language each do one job well.

4

Putting It Together

Choosing, evaluating, and building

Which Model for Which Job?

No single winner — match tool to task

1

Lots of side features, sparse data

Factorization Machine or DeepFM — built for rich, sparse feature sets like click prediction.

2

Millions of items, real-time serving

Two-Tower for retrieval, then a heavier ranker (DeepFM). The standard two-stage system.

3

Order matters / explanation needed

SASRec for life-stage sequences; LightGCN for sparse graphs; an LLM when you must explain WHY.

Start simple (FM), add deep parts only where they earn their keep. Same discipline as Week 1.

Deeper Models, Same Old Discipline

Power raises the stakes, not the rules

A deep recommender can overfit spectacularly and still look great in an offline test.

Everything from Part 1 and Week 10 still holds — maybe more so, because these models have more ways to cheat.

Evaluate honestly

- Split by TIME, not at random (no peeking ahead).
- Use precision@k / recall@k on held-out data.
- Watch for popularity bias and feedback loops.

Be responsible (Week 14)

- Explain recommendations — especially for money.
- Check fairness across customer groups.
- Don't trap people in a narrow bubble of offers.

Deep Recommenders in Finance

Where each model fits

Retail banking

- DeepFM: predict next-product clicks.
- Two-Tower: real-time offers to millions.
- SASRec: 'savings to fund to insurance' journeys.

Wealth & advice

- GNN: cross-sell via the client-product graph.
- LLM: explain a fund suggestion in plain words.
- Blend for accuracy + transparency.

Always remember

- Stakes are someone's savings.
- Honesty and fairness beat raw accuracy.
- Keep a human in the loop for big decisions.

Same engine as Big Tech — but applied with the care that money demands.

Getting Started

Tiny inline demos — see the mechanisms, not a framework

Built-in toy data (no download)

- Real deep recs need big libraries and GPUs.
- So we capture the CORE IDEA in a few numpy lines.
- Lab 1: an FM-style feature-interaction score.
- Lab 2: a Two-Tower-style dot-product match.

What to take away

- You won't train a production model today.
- But you'll SEE why FM and Two-Tower work.
- That intuition transfers to the real libraries.
- Scale up later with LibRecommender, RecBole, etc.

colab_setup.py — tiny inline demo (no download)

```
import numpy as np
rng = np.random.default_rng(0)
# 3 one-hot-ish features: [is_young, is_evening, is_mobile]
x = np.array([1.0, 1.0, 0.0]) # a young user, evening, not mobile
```

Hands-On Lab 1 — Factorization Machine Score

Feature interactions via hidden vectors

Your task (step by step)

- 1. Give each feature a small hidden vector.
- 2. Add each feature's own weight (linear part).
- 3. Add every PAIR's interaction (dot products).
- 4. That sum is the FM score.
- Question: how do interactions change the score?

`colab_lab1.py` • [tiny inline demo \(no download\)](#)

```
import numpy as np
x = np.array([1.0, 1.0, 0.0])      # active features
w0, w = 0.1, np.array([0.2, -0.1, 0.3]) # bias + linear
V = np.array([[0.5, 0.1],         # hidden vector per
               [0.2, 0.4],         # feature (k=2)
               [0.9, 0.0]])

linear = w0 + w @ x
inter = sum((V[i]@V[j]) * x[i]*x[j]
            for i in range(3) for j in range(i+1, 3))
print('FM score:', round(linear + inter, 3))
```

Hands-On Lab 2 — Two-Tower Matching

User vector dot item vectors -> best match

Your task (step by step)

- 1. Make one user vector (the user tower output).
- 2. Make several item vectors (the item tower).
- 3. Score by dot product (the match).
- 4. Recommend the highest-scoring item.
- Question: which fund matches this user best?

colab_lab2.py · tiny inline demo (no download)

```
import numpy as np
# user tower output (one vector)
user = np.array([0.8, 0.2, 0.5])
# item tower outputs (one row per fund)
items = np.array([[0.7, 0.1, 0.6], # fund 0
                  [0.1, 0.9, 0.0], # fund 1
                  [0.9, 0.0, 0.4]]) # fund 2

scores = items @ user # dot-product match
best = scores.argmax()
print('match scores:', scores.round(2))
print('recommend fund', best)
```

Comprehension Check

Try first, then reveal the answer below

Q1 Why is the Factorization Machine called the 'bridge'?

- A It is the fastest model
- B It generalizes MF to all features and leads into DeepFM ✓
- C It only works on images
- D It replaces neural nets

Answer: FM extends matrix factorization to any feature (not just IDs); DeepFM then adds a neural net — the step into deep learning.

Q2 Why does the Two-Tower model scale to millions of items?

- A It uses no math
- B Item vectors are precomputed; only fast nearest-neighbor search runs live ✓
- C It ignores users
- D It needs no training

Answer: Item embeddings are computed once and stored; at request time a fast nearest-neighbor search (Faiss) finds top matches.

The Ladder, Top to Bottom

1

FM & DeepFM — the bridge

FM generalizes MF to all features and learns pairwise interactions; DeepFM adds a deep net for high-order patterns.

2

NCF, Two-Tower, SASRec

Non-linear taste (NCF), scalable retrieval (Two-Tower), and order-aware life-stage prediction (SASRec).

3

GNN & LLM frontier

LightGCN/ContextGNN on the customer-product graph; LLMs that recommend and explain. Hybrids lead.

It's all your toolkit recombined: factors (W9), nets (W8), attention & LLMs (W12), graphs (W15).

Part 2 — Three Things to Remember

1

Climb the ladder, don't leap

MF to FM to DeepFM to deep. FM is the bridge: same dial idea, every feature, all pairwise interactions.

2

Each deep model has a job

NCF for non-linearity, Two-Tower for scale, SASRec for sequence, GNN for sparse graphs, LLM for explanation.

3

More power, same discipline

Split by time, evaluate honestly (Week 10), explain and stay fair (Week 14). Deeper means more ways to fool yourself.

Part 1 + Part 2 together: from the classics to the frontier of recommendation.

Try both labs, then explore a real library (RecBole, LibRecommender) on a public ratings dataset.